

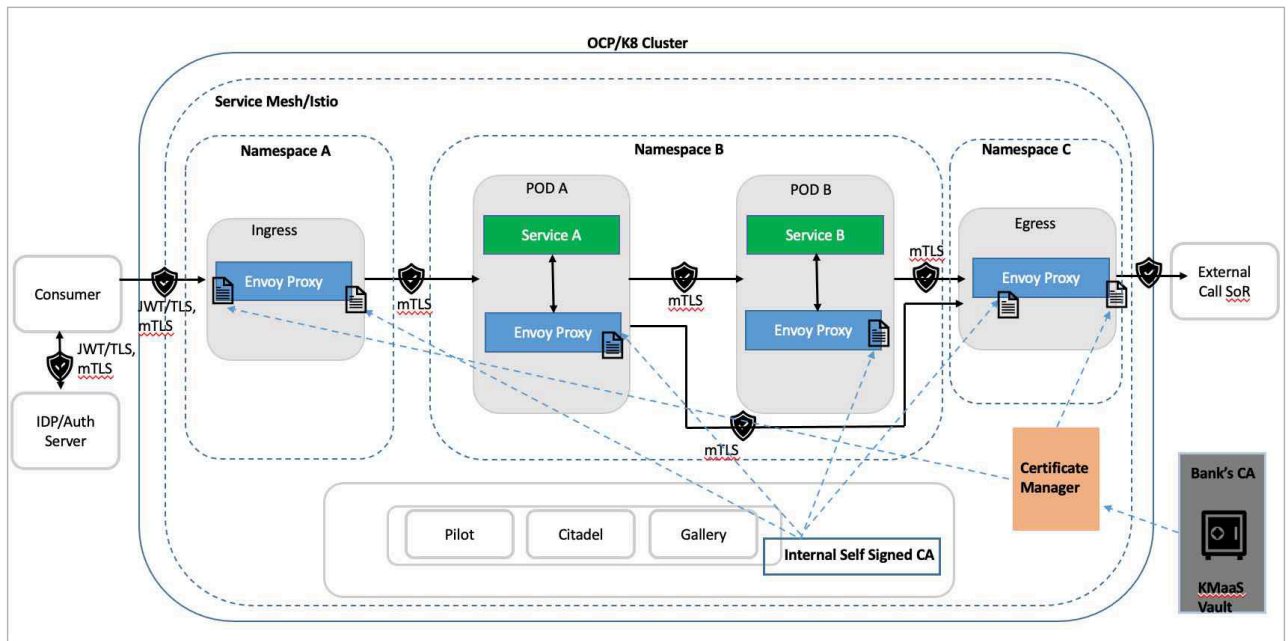


Figure 6: Security within service mesh

Security area: Authorisation

Solution: Authorisation policy should restrict access to the external system by validating the scope in access token. Claim and signature in JWT also needs to be validated to ensure the token can be used.

Application: This needs integration with enterprise identity provider (IDP).

Security area: Cross origin resource sharing (CORS)

Solution: If CORS is required, it should be implemented in service mesh instead of application code.

Application: None

Security area: Managed egress

Solution: Service mesh should be configured with service registry, which should contain the external services that should be accessed.

Application: Set mesh-wide *outBoundTrafficPolicy* to *REGISTRY_ONLY* in the mesh to ensure no service call from service mesh bypasses egress to reach any external system.

Security area: Access logging

Solution: Access to pods and gateways should be logged, along with any non-200 responses to detect attack.

Application: Envoy sidecars should be configured to record standard message information in the access log including time of request, method, protocol, upstream host, authority, and details of access token.

Impact of the pattern

- Consuming application will need to provide an access token signed by an authorisation server trusted by service mesh.

- The performance impact of taking a zero trust approach and validating a token in each side car in the mesh needs to be factored in solution architecture and testing.

Service/container deployment

The problem it solves

Deployment of service containers in the production environment of any large enterprise carries the risk of code behaving differently in production environments and higher test environments. One common example is where production-specific environment variables substituted in the build pipeline do not work properly and are only discovered when the new deployment is live. This risk is coupled with the demand for a rapid deployment cadence to deliver changes to the production environment.

Pattern description and solution

Service mesh addresses this problem by simplifying traffic management and enhancing deployment strategies such as canaries, dark launches and traffic mirroring. Each provides a mechanism to mitigate risks in production changes by separating code release from cutover and a degree of testing in production. Each of the strategies is described briefly below.

Canary deployment: This mitigates the risks associated with production changes by routing a subset of requests of the traffic to the new version while the remaining traffic is routed to the existing version. For example, for a service *ViewPolicyDetails* in production with version 1.0, a new